

# WEB TASARIMINA GİRİŞ: HTML VE TEMEL BİLGİLER



- Web Tasarımında Temel Kavramlar
- Html: Web Sayfalarının Temel Taşı
- Semantik ve Erişilebilirlik



**Atatürk Üniversitesi**  
Açıköğretim Fakültesi

## WEB TASARIMININ TEMELLERİ Doç. Dr. Murat ÇOBAN



- Bu üniteyi çalıştıktan sonra;
  - Web tasarımının temel kavramlarını, kullanıcı odaklı ve çağdaş yaklaşımlar ışığında kavrayabilecek,
  - Front-end (ön yüz) ve back-end (arka yüz) kavramlarını ayırt ederek, ön yüz geliştirmedeki araçları ve editörleri etkin kullanabilecek,
  - Web tasarımında kullanılan temel araçlar hakkında bilgiler edinecek,
  - HTML, CSS ve JavaScript'in hakkında temel bilgiler edineceksiniz.

## ÜNİTE 4

## GİRİŞ

Günümüzde dijital dünyada var olmanın en önemli yollarından biri, etkili ve kullanıcı dostu web sitelerine sahip olmaktır. İnternet kullanımının artmasıyla birlikte hem bireyler hem de kurumlar kendilerini daha iyi ifade etmek ve geniş kitlelere ulaşmak için web sitelerine ihtiyaç duymaktadır. Bu ihtiyaçların artmasıyla beraber, web tasarımı kavramı da giderek daha fazla önem kazanmaktadır. Etkili bir web sitesine sahip olmak, dijital dünyada ilk izlenimi oluşturmanın anahtarı olduğu söylenebilir.

Başarılı bir web sitesi oluşturmanın ilk adımı, web tasarımının temellerini anlamaktan geçmektedir [1]. Temel kavramlar arasında sitenin yapısı, kullanılabilirliği ve görsel uyumu önemli bir yer tutar. Kullanıcıların siteyi rahatça gezebilmesi için düzenli bir yapı oluşturmak, doğru renk seçimi ve tipografiyle estetik bir görünüm elde etmek temel unsurların başında gelir. Ayrıca, sitenin farklı cihazlarda sorunsuz çalışabilmesi ve hızlı yüklenmesi günümüz kullanıcıları için vazgeçilmezdir. Bu noktada, kullanıcı deneyimi ve kullanıcı arayüzü tasarımı, modern web projelerinin temel taşları olarak öne çıkmaktadır. Bu bağlamda web projelerinin arayüz tasarımında kullanılan araçlar da önem kazanmıştır.

Web teknolojinin gelişimiyle beraber bahsi geçen arayüz tasarım araçları da sürekli gelişmekte ve değişmektedir. Temel olarak HTML, bir web sayfasının iskeletini oluştururken; *CSS*, web sitesinin görünümünü şekillendirir ve estetik kazandırır [4]. *JavaScript* ise, dinamik ve etkileşimli arayüzler oluşturulmasını mümkün kılar. Günümüzde ise bu teknolojilere ek olarak; *React*, *Vue*, *Angular* gibi gelişmiş *JavaScript* kütüphaneleri ile daha hızlı ve dinamik siteler geliştirmek mümkündür. Dijital tasarım tarafında ise *Figma* ve *Adobe XD* gibi araçlar, ekiplerin iş birliği içinde çalışmasını kolaylaştırır ve tasarımdan kodlamaya geçiş sürecini pratikleştirir.

Güncel kullanıcı beklentileri çerçevesinde web sitelerinin mobil uyumluluğu da oldukça önemli hale gelmiştir. Google'ın mobil öncelikli indeksleme stratejisi, sitelerin önce mobilde nasıl çalıştığına göre değerlendirilmesini sağlar. Bunun yanı sıra, açık ve koyu tema seçenekleri, animasyonlar, mikro etkileşimler ve hızlı yükleme süreleri de modern web tasarımının ayrılmaz parçaları arasında yer alır. Ayrıca, erişilebilirlik (accessibility) standartlarını gözetten siteler, engelli bireyler için de eşit dijital erişim imkânı sağlayarak yasal ve etik sorumlulukları karşılar.

Son yıllarda yapay zekâ destekli tasarım araçları ve otomasyon çözümleri de web tasarımının gelişimine önemli katkılar sunmaktadır. Örneğin, ChatGPT gibi yapay zekâ tabanlı araçlar sayesinde hızlıca farklı biçimlerde temalar üretilmekte veya kod önerileri alınabilmektedir [3]. Bu gelişmeler hem zamandan tasarruf sağlamayı hem de daha erişilebilir ve yenilikçi içeriklerin oluşturulmasını mümkün kılmaktadır. Kısacası, web tasarımının temelleri, yalnızca görsel bir yapı oluşturmakla kalmaz; kullanıcı ihtiyaçlarını analiz etmeyi, en güncel teknolojileri takip etmeyi ve erişilebilir, hızlı, modern arayüzler geliştirmeyi de kapsar. Başarılı ve güncel bir web sitesi kurmak isteyen herkes için bu temel kavramları ve araçları bilmek artık bir gerekliliktir.



**CSS (Cascading Style Sheets)**, web sayfalarının görsel sunumunu ve biçimlendirilmesini sağlamak için kullanılan bir stil dilidir. HTML ile birlikte çalışarak, web sayfalarının estetik görünümünü belirler.



JavaScript, web teknolojileriyle etkileşimli ve dinamik uygulamalar geliştirmek için vazgeçilmez bir araçtır. Geliştiricilere geniş bir uygulama yelpazesi sunarak, modern web deneyimlerinin temel bileşenlerinden biri haline gelmiştir.

Bu bölümün amacı, okuyucuya web tasarımına dair temel kavramları ve güncel gelişmeleri aktararak sağlam bir başlangıç noktası sunmaktır. Önümüzdeki başlıklarda modern web sitelerinin planlanması, kullanılan tasarım yaklaşımları, *HTML, CSS ve JavaScript* gibi teknolojilerin rolü ile erişilebilirlik ve mobil uyumluluk gibi güncel trendlerin önemi ayrıntılı bir şekilde ele alınacaktır. Böylece, web sitesinin temellerinden başlayarak günümüz standartlarına uygun, etkili ve kullanıcı dostu bir tasarımın nasıl oluşturulacağı hakkında kapsamlı bir bakış açısına sahip olabileceksiniz.

## Web Tasarımında Temel Kavramlar

Web tasarımı, yalnızca bir internet sayfasının görsel olarak güzel görünmesiyle sınırlı değildir; aynı zamanda kullanıcıların ihtiyaçlarına cevap veren, erişilebilir, işlevsel ve etkili bir dijital deneyim sunmayı da hedefler. Bu amaçla, başarılı bir web sitesi oluşturmada göz önünde bulundurulması gereken bazı temel kavramlar vardır. Her şeyden önce, kullanıcı deneyimi (UX) ve kullanıcı arayüzü (UI) kavramları öne çıkar. Kullanıcı deneyimi, siteyi ziyaret edenlerin beklentilerini karşılayan, rahat ve sezgisel bir gezinme imkânı sağlayan bütüncül bir deneyimin sağlanmasına odaklanır. Kullanıcı arayüzü ise, görsel düzen, renkler, tipografi, butonlar gibi unsurların doğru ve uyumlu şekilde yerleştirilmesini içerir. Bir web sitesi ne kadar estetik olursa olsun, eğer kullanıcı istediği bilgiye kolayca ulaşamıyorsa veya gezinirken zorlanıyorsa, etkili bir tasarımdan söz etmek mümkün değildir.

Site yapısı ve navigasyon web tasarımının bel kemiğidir. İyi planlanmış bir site haritası ve mantıklı menü düzenleriyle kullanıcı, aradığı içeriğe kısa sürede ulaşabilir. Karmaşık, gereksiz detaylarla dolu menüler, ziyaretçilerin siteyi terk etmesine neden olabilir. Bu yüzden sade, anlaşılır ve işlevsel bir yapı tasarlamak her zaman önceliklidir. Erişilebilirlik (Accessibility), günümüzde dijital dönüşümün en önemli gerekliliklerinden biri haline gelmiştir. Her kullanıcının, herhangi bir engeli olup olmadığı gözetmeksizin siteye rahatça erişebilmesi gerekir. Renk körü kullanıcılar için doğru renk kontrastları, ekran okuyucu uyumu gibi unsurlar, sitenin daha kapsayıcı olmasını sağlar. Responsive (duyarlı) tasarım da temel kavramlar arasında yer alır. Mobil cihazlar üzerinden web'e erişimin her geçen gün arttığı günümüzde, bir sitenin farklı ekran boyutlarında sorunsuz çalışması şarttır. Responsive tasarım sayesinde, web siteleri masaüstü, tablet ve telefon gibi cihazlarda uyumlu görünür ve tüm fonksiyonlarını korur [2].

Web tasarımında öne çıkan diğer temel kavramlar arasında hız ve performans yer alır. Yavaş yüklenen siteler, ziyaretçileri kaybetmenize sebep olabilir. Görsel optimizasyonları sağlamak, gereksiz kodlardan kaçınmak ve güvenilir sunucular kullanmak, sitenin hızını ve kullanıcıların memnuniyetini artırır. Son olarak, içerik ve SEO (Arama Motoru Optimizasyonu) tasarımın ayrılmaz parçalarıdır. Anlamlı, özgün ve hedef kitleye uygun içerikler, kullanıcıların siteyle etkileşime girmesini sağlar. Ayrıca doğru anahtar kelimeler ve yapısal düzenlemelerle arama motorlarında üst sıralarda yer almak mümkün olur. Tüm bu temel kavramlar, bir web sitesini yalnızca görsel olarak değil, aynı zamanda

işlevsel ve erişilebilir hâle getirmek için birlikte çalışır. Günümüzün rekabetçi dijital dünyasında kalıcı ve etkili bir web varlığı oluşturmak isteyen herkes için bu kavramlara hâkim olmak büyük bir avantajdır.

## Front-end ve Back-end Kavramları



Front-end sürecinde özellikle CSS yapılarından yararlanılmaktadır.

Front-end (Ön Uç), bir web sitesinde veya uygulamada kullanıcının doğrudan etkileşimde bulunduğu kısımdır. Yani, ziyaretçilerin tarayıcıda gördüğü ve kullandığı arayüzlerdir. Front-end aşamasında genellikle HTML, CSS ve JavaScript gibi kodlama yapıları kullanılarak web sayfasının görsel arayüz kısmı tasarlanmaktadır. Örneğin, alışveriş sitesinin anasayfasında gördüğünüz menüler, ürün görselleri, sepete ekle butonları, arama çubuğu gibi öğeler front-end olarak adlandırılan kodlama süreci içerisinde tasarlanmaktadır.

Şekil 4.1’de Visual Studio Code editöründe yazılmış basit bir Front-end kod örneğini inceleyebilirsiniz. Kullanıcı buradaki kodlar sayesinde oluşturulan butona tıklar ve bir mesaj alır.

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Frontend Örneği</title>
5    <style>
6      button { background-color: #28a745; color: white; padding: 10px; border: none; }
7    </style>
8  </head>
9  <body>
10   <h1>Merhaba, kullanıcı!</h1>
11   <button onclick="gosterMesaj()">Tıkla</button>
12   <script>
13     function gosterMesaj() {
14       alert('Butona tıklandı!');
15     }
16   </script>
17 </body>
18 </html>
19 <!DOCTYPE html>
20 <html>
21 <head>
22   <title>Frontend Örneği</title>
23   <style>
24     button { background-color: #28a745; color: white; padding: 10px; border: none; }
25   </style>
26 </head>
27 <body>
28   <h1>Merhaba, kullanıcı!</h1>
29   <button onclick="gosterMesaj()">Tıkla</button>
30   <script>
31     function gosterMesaj() {
32       alert('Butona tıklandı!');
33     }
34   </script>
35 </body>
36 </html>

```

Şekil 4.1. Front-end Kod Örneği

Back-end (Arka Uç), uygulamanın kullanıcının görmediği, sunucu tarafında çalışan kısımdır. Bu kodlama sürecinde veritabanı işlemleri, kullanıcı doğrulama, sunucu işlemleri, gibi pek çok işlem yapılmaktadır. Bu işlemlerde özellikle Python (Django, Flask), JavaScript (Node.js), PHP, Java ve Ruby gibi diller kullanılmaktadır.

Bir kullanıcı giriş yaparken arayüze e-posta ve şifre girer (front-end), ancak bu bilgiler sunucuya gönderilir, orada kontrol edilip kullanıcıya “Giriş

başarılı/başarısız” diyerek cevap verilir (back-end). Şekil 4.2’de bu süreci anlatan bir JavaScript diliyle yazılmış kod örneği sunulmaktadır.

```

web.html JS web.js
JS web.js > app.listen() callback
1 // Node.js ve Express ile backend örneği
2 const express = require('express');
3 const app = express();
4
5 app.use(express.json());
6
7 app.post('/login', (req, res) => {
8   const { email, password } = req.body;
9   if(email === 'kullanici@site.com' && password === '12345') {
10    res.send('Giriş başarılı!');
11   } else {
12    res.status(401).send('Hatalı bilgiler!');
13   }
14 });
15
16 app.listen(3000, () => {
17   console.log('Sunucu 3000 portunda çalışıyor.');

```

Şekil 4.2. JavaScript Dilinde Kodlanmış Bir Back-end Yapısı



Front-end ve Back-end yapıları birlikte kullanılarak web projelerinin çalışır hali ortaya çıkmaktadır.

Front-end yapıları kullanıcıya yönelik, görsel ve etkileşimli kısımları içermektedir. Back-end yapıları ise sunucu tarafında mantık ve veri yönetimi işlemlerini içermektedir. Back-end kodlarında genellikle “API” adı verilen arabirimler kullanılır.

Front-end, kullanıcıdan aldığı verileri back-end’e yollar ve oradan gelen cevaba göre hareket eder. Daha somut bir ifadeyle Front-end bir aracın kaportası olarak düşünülebilir. Back-end ise arabayı hareket ettiren motor aksamı olarak düşünülebilir.

## Web Tasarımında Kullanılan Bazı Araçlar ve Editörler

Web tasarımı, yalnızca estetik öğelerin değil, aynı zamanda kullanıcı deneyimi, erişilebilirlik ve teknik uygulamanın birlikte ele alındığı multidisipliner bir alandır. Web tasarım süreçlerinde gerek arayüz geliştirme gerekse grafik ve prototip hazırlama aşamalarında birçok araç ve editör aktif şekilde kullanılmaktadır. Bu araçlar, tasarımcı ve geliştiricilerin üretkenliğini artırmak, hata oranını azaltmak ve ortaya çıkan ürünün kalitesini yükseltmek amacıyla tasarlanmıştır.

Web sitelerinde yer alan görsel öğelerin ve arayüz tasarımlarının hazırlanmasında grafik ve tasarım araçları kritik rol oynamaktadır. Özellikle Front-end sürecinde bu araçlardan fazlasıyla yararlanılmaktadır. Aşağıda yaygın olarak kullanılan bazı grafik ve tasarım araçları sunulmaktadır:

Adobe Photoshop: Profesyonel düzeyde grafik ve görüntü işleme yazılımları arasındadır. Web arayüzlerinin ve görsel materyallerin hazırlanmasında yaygın olarak kullanılmaktadır.

<https://www.adobe.com/products/photoshop.html>

- **Adobe XD:** UI/UX tasarımı ve prototipleme alanında öne çıkan bir araçtır. Takım çalışmasına uygun olarak geliştirilmiştir.  
<https://www.adobe.com/products/xd.html>
- **Figma:** Bulut tabanlı olarak çalışan Figma, birlikte çalışma ve paylaşım imkân verir. Özellikle UI/UX tasarımcıları arasında oldukça popülerdir.  
<https://www.figma.com/>
- **Sketch:** Sadece macOS platformunda çalışan bir vektörel tasarım aracıdır ve özellikle arayüz (UI) tasarımında tercih edilmektedir.  
<https://www.sketch.com/>
- **Canva:** Teknik bilgisi az olan bireylerin de hızlıca görsel ve grafik tasarım yapmalarına fırsat tanıyan bir yazılım olarak öne çıkmaktadır. Ayrıca web tabanlı ve geniş şablon yelpazesine sahip bir araç olarak öne çıkmaktadır.  
<https://www.canva.com/>

Kod editörleri, web tasarımının temel bileşenlerini oluşturan HTML, CSS, JavaScript ve diğer programlama dillerinin yazılması ve yönetimi için kullanılmaktadır. Ayrıca front-end tarında kullanılan bu editörler back-end tarafında da kullanılmaktadır. Aşağıda yaygın olarak kullanılan kod editörleri ve bağlantı adresleri sunulmaktadır:

- **Visual Studio Code (VS Code):** Microsoft tarafından geliştirilen bu editör, eklenti desteği ve modern arayüzüyle öne çıkmaktadır. Şekil 1.2’de Visual Studio Code editöründe yazılmış örnek kodlar görülebilir. Kullanıcıların hizmetine ücretsiz olarak sunulan bu editöre aşağıdaki bağlantıdan ulaşılabilir.  
<https://code.visualstudio.com/>
- **Sublime Text:** Hafifliği ve hızının yanı sıra sunduğu geniş eklenti ekosistemi ile bilinmektedir.  
<https://www.sublimetext.com/>
- **Atom:** GitHub tarafından geliştirilen ve açık kaynak kodlu bir yapıya sahip editördür. Bu editör özelleştirilebilirliği ile tercih edilmektedir.  
<https://atom.io/>
- **Notepad++:** Özellikle Windows platformunda yaygın kullanılan, hafif ve pratik bir editördür.  
<https://notepad-plus-plus.org/>

Bu temel araçlara ek olarak web tasarımı sürecinde prototipleme araçları, web tarayıcı geliştirici araçları, sürüm kontrol ve ekip çalışması araçları da eklenebilir. Örneğin bir web projesinin ilk aşamalarında, sayfa düzeninin ve



Kod editörleri sayesinde web projelerine kodlar eklenebilir ve bu odların etkileri gözlemlenebilir. Kod editörleri arasında VS Code yaygın olarak kullanılmaktadır.

kullanıcı akışının belirlenmesinde Balsamiq Mockups, Figma & Adobe XD gibi işlevsel araçlar kullanılabilir. Kodların doğruluğunun kontrolü, performans analizi ve hata ayıklama işlemlerinde ise web tarayıcılarının geliştirici araçları aktif olarak kullanılmaktadır. Aşağıda bazı geliştirici araçlar ve bağlantı adresleri sunulmaktadır:

- **Google Chrome Developer Tools:**  
<https://developer.chrome.com/docs/devtools/>
- **Firefox Developer Tools:**  
<https://firefox-source-docs.mozilla.org/devtools-user/>
- **Safari Developer Tools:**  
<https://developer.apple.com/safari/tools/>

Bazı web projeleri genellikle ekip halinde yürütülmektedir. Bu bağlamda kod yönetimi için sürüm kontrol sistemleri de kullanıcılara büyük kolaylıklar sunmaktadır. Aşağıda bazı sürüm kontrol sistemleri sunulmaktadır:

- **Git:**  
<https://git-scm.com/>
- **GitHub:**  
<https://github.com/>
- **GitLab:**  
<https://gitlab.com/>
- **Bitbucket:**  
<https://bitbucket.org/>



Web projelerinde özellikle WordPress gibi hazır araçlar sayesinde hızlı ve etkili tasarımlar yapılabilir.

Son olarak kodlama bilgisi kısıtlı olan veya hızlıca web sitesi yayına almak isteyenler için hazır sistemlerin sunulduğu araçlarda bulunmaktadır. Özellikle hazır tasarım şablonlarının ve eklentilerin olduğu bu tür araçlardan yararlanarak web sayfaları kolaylıkla tasarlanabilmektedir. Yaygın olarak kullanılan bu tür araçlar arasında WordPress aracının olduğu söylenebilir. Aşağıdaki adresten bu araçla ilgili detaylı bilgilere ulaşılabilir.

- **WordPress:**  
<https://wordpress.org/>

Sonuç olarak web tasarımında kullanılan araç ve editörler, her geçen gün gelişerek ve çeşitlenerek sürecin her aşamasında farklı uzmanlık alanlarına çözümler sunmaktadır. Araç seçiminde, projenin gereksinimleri, bireysel ya da ekip çalışma dinamikleri ve kullanıcı beklentileri belirleyici olmaktadır. Doğru ve güncel araçlar ile nitelikli web tasarımları üretmek çok daha kolay ve verimli hale gelmektedir.

## HTML: WEB SAYFALARININ TEMEL TAŞI

**HTML (HyperText Markup Language)**, web sayfalarının iskeletini oluşturan temel işaretleme dilidir. Her türlü içeriğin (metin, görsel, bağlantı veya tablonun) tarayıcıda düzgün şekilde görüntülenmesini sağlayan yapı taşlarını sunar. İnternet üzerinde gördüğümüz sitelerin çoğunun arkasında, bilgiyi düzenli ve erişilebilir kılan HTML bulunur. Bu nedenle HTML, modern web tasarım ve geliştirme süreçlerinin vazgeçilmez temel unsurudur. HTML dilinde yaygın olarak kullanılan etiketler ve işlevleri örneklerle birlikte aşağıdaki gibi sunulmaktadır:

### Temel Yapı

Bir HTML dokümanının temel yapısı, **başlık ve gövde** olmak üzere iki ana bölümden oluşmaktadır. Başlık bölümünde sayfa başlığı ve meta bilgiler, gövde bölümünde ise ziyaretçilere sunulan asıl içerik yer alır. Her içerik elemanı, belirli bir görevi olan özel etiketlerle tanımlanır. HTML’de en sık karşılaşılan etiketler arasında `<html>`, `<head>`, `<body>`, `<h1>`-`<h6>`, `<p>`, `<a>`, `<img>`, `<ul>`, `<ol>`, `<li>`, `<table>`, `<tr>`, `<td>`, `<th>`, `<div>`, `<span>`, `<form>`, `<input>` ve `<button>` gibi temel yapı taşları bulunur.

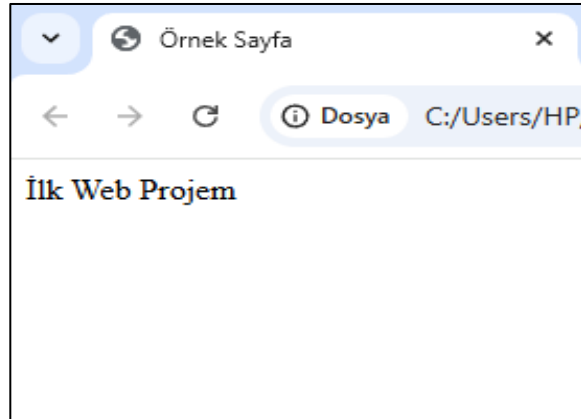
```

<> web.html > html
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <title>Örnek Sayfa</title>
5    </head>
6    <body>
7      İlk Web Projem
8    </body>
9  </html>

```

Şekil 4.3. Temel HTML Sayfası Kod İçeriği

Şekil 4.3’teki kodu bir not defterine veya *Visual Studio Code* editörü gibi bir editöre yazdıktan sonra dosyanın uzantısı “*html*” olacak biçimde kaydedilebilir. Daha sonra bir tarayıcı ile bu dosya açıldığında veya kod editörüyle çalıştırıldığında Şekil 4.4’teki ekran görüntüsüyle karşılaşılacaktır.



Şekil 4.4. İlk HTML kodları Çalıştırıldığında Ortaya Çıkan Tablo

<!DOCTYPE html>, bir HTML dosyasının en üst satırında yer alan ve tarayıcıya bu belgenin hangi HTML sürümünde yazıldığını belirten özel bir bildiridir.

Modern web geliştirmede kullanılan <!DOCTYPE html>, sayfanın HTML5 standardına uygun olduğunu belirtir ve tarayıcıların sayfayı doğru şekilde yorumlamasını sağlar. Bu bildirim sayfa kaynağının bir parçası değildir. Ancak eksik olursa tarayıcılar bazı eski (uyumluluk) modlarına geçebilir ve sayfa beklenildiği gibi görüntülenmeyebilir.

## Temel HTML Etiketleri

Temel HTML etiketleri, bir web sayfasının yapısını ve içeriğini tanımlayan yapı taşlarıdır. Başlıklar, paragraflar, listeler, bağlantılar ve görseller gibi elementler hem tarayıcıya hem de arama motorlarına içeriğin anlamını iletir. HTML ile ilgili temel etiketler ve kullanım örnekleri aşağıdaki gibi sunulmaktadır:

### <html> Etiketi

Belgenin kök (root) ögesidir. Tüm HTML içeriği bunun içinde olmalıdır. Aşağıda <html> etiketine ilişkin örnek bir kod yapısı Şekil 4.5'te sunulmaktadır:

```
<!DOCTYPE html>
<html lang="tr">
  <!-- head ve body buraya gelir -->
</html>
```

Şekil 4.5. <html> Etiketine İlişkin Örnek Bir Kod Yapısı

### <head> Etiketi

Sayfa meta verileri, başlık, stil ve script bağlantıları buraya konur. Örneğin, <title> etiketi sayesinde sayfanın başlık bilgisi tanımlanmıştır. <link> etiketi sayesinde de “*styles.css*” adlı bir CSS dosyasının biçimlendirme özellikleri referans alınmıştır.

Başka bir ifadeyle hazırlayacağınız web projesine ait dosyalarda “*styles.css*” adlı bir dosyanın önceden hazırlanmış olması gerekmektedir. Bu kodların örnek kullanımı Şekil 4.6’da sunulduğu gibidir:

```
<head>
  <meta charset="utf-8">
  <title>Örnek Sayfa</title>
  <link rel="stylesheet" href="styles.css">
</head>
```

Şekil 4.6. <head> Etiketine İlişkin Örnek Bir Kod Yapısı



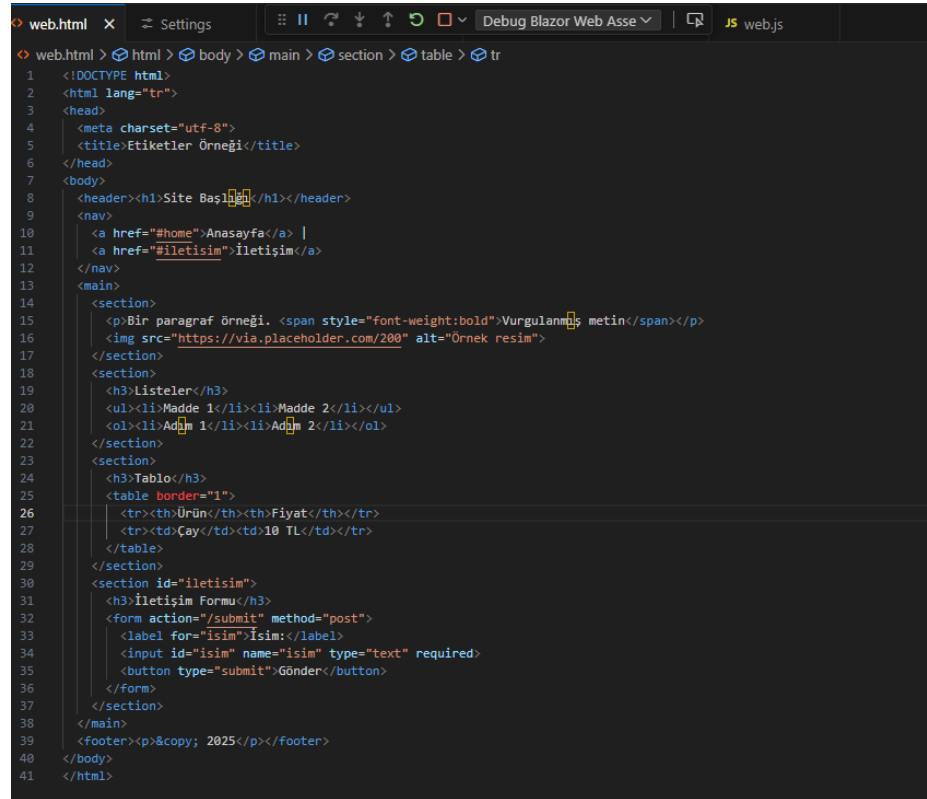
Web projelerinde özellikle sayfa bölümlerinde <div> etiketinden yararlanılmaktadır.

Tablo 1.1’de temel HTML etiketlerini, kısa açıklamalarını ve birer kısa örnek kod satırını tablo halinde bulabilirsiniz.

| <b>Tablo 1.1. Temel HTML Etiketlerine İlişkin Örnekler</b> |                                                                        |                                                                                                    |
|------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <b>Etiket</b>                                              | <b>Kısa Açıklama</b>                                                   | <b>Örnek Kod</b>                                                                                   |
| <code>&lt;!DOCTYPE html&gt;</code>                         | Belge tipini ve HTML5 kullanıldığını belirtir.                         | <code>&lt;!DOCTYPE html&gt;</code>                                                                 |
| <code>&lt;html&gt;</code>                                  | Belgenin kök ögesi; tüm içerik bunun içinde yer alır.                  | <code>&lt;html lang="tr"&gt;...&lt;/html&gt;</code>                                                |
| <code>&lt;head&gt;</code>                                  | Meta veriler, başlık, stil/script bağlantıları yer alır.               | <code>&lt;head&gt;&lt;meta charset="utf-8"&gt;&lt;title&gt;Sayfa&lt;/title&gt;&lt;/head&gt;</code> |
| <code>&lt;body&gt;</code>                                  | Görünen tüm içerikler (metin, görsel, form vb.) bu etiket arasındadır. | <code>&lt;body&gt;...&lt;/body&gt;</code>                                                          |
| <code>&lt;h1&gt;</code> - <code>&lt;h6&gt;</code>          | Başlık etiketleri için kullanılır.                                     | <code>&lt;h1&gt; en yüksek öncelik.&lt;/h1&gt; Sayfa Başlığı&lt;/h1&gt;</code>                     |
| <code>&lt;p&gt;</code>                                     | Paragraf metni için kullanılır.                                        | <code>&lt;p&gt;Bu bir paragraftır.&lt;/p&gt;</code>                                                |
| <code>&lt;a&gt;</code>                                     | Köprü (link) eklemek için kullanılır.                                  | <code>&lt;a href="https://example.com"&gt;Site&lt;/a&gt;</code>                                    |
| <code>&lt;img&gt;</code>                                   | Görsel eklemek için kullanılır.                                        | <code>&lt;img src="img.jpg" alt="Açıklama"&gt;</code>                                              |
| <code>&lt;ul&gt;</code>                                    | Sırasız (madde işaretli) liste kapsayıcı eklemek için kullanılır.      | <code>&lt;ul&gt;&lt;li&gt;Öğe&lt;/li&gt;&lt;/ul&gt;</code>                                         |
| <code>&lt;ol&gt;</code>                                    | Sıralı (numaralı) liste kapsayıcı eklemek için kullanılır.             | <code>&lt;ol&gt;&lt;li&gt;Adım 1&lt;/li&gt;&lt;/ol&gt;</code>                                      |
| <code>&lt;li&gt;</code>                                    | Liste ögesi; ul veya ol içinde olacak biçimde kullanılabilir.          | <code>&lt;li&gt;Liste ögesi&lt;/li&gt;</code>                                                      |
| <code>&lt;table&gt;</code>                                 | Tablo kapsayıcısı; satırlar ve hücreler içerir.                        | <code>&lt;table&gt;&lt;tr&gt;&lt;td&gt;Veri&lt;/td&gt;&lt;/tr&gt;&lt;/table&gt;</code>             |
| <code>&lt;tr&gt;</code>                                    | Tablo satırı (row).                                                    | <code>&lt;tr&gt;&lt;td&gt;Hücre1&lt;/td&gt;&lt;td&gt;Hücre2&lt;/td&gt;&lt;/tr&gt;</code>           |
| <code>&lt;td&gt;</code>                                    | Tablo veri hücresi (normal hücre).                                     | <code>&lt;td&gt;Veri&lt;/td&gt;</code>                                                             |
| <code>&lt;th&gt;</code>                                    | Tablo başlık hücresi (header cell).                                    | <code>&lt;th&gt;Başlık&lt;/th&gt;</code>                                                           |

|          |                                                                     |                                                 |
|----------|---------------------------------------------------------------------|-------------------------------------------------|
| <div>    | Genel amaçlı blok seviyesinde kapsayıcı (semantik değil).           | <div class="card">İçerik</div>                  |
| <span>   | Satır içi genel amaçlı kapsayıcı                                    | <p>Bu <span>virgulu</span> metin.</p>           |
| <form>   | Kullanıcıdan veri toplamak için form kapsayıcısı olarak kullanılır. | <form action="/submit" method="post">...</form> |
| <input>  | Tek satırlık giriş alanı; type ile davranış belirlenir.             | <input type="email" id="e" name="e">            |
| <button> | Buton; form gönderme veya JavaScript etkileşimi için kullanılır.    | <button type="submit">Gönder</button>           |

Bütün bu kodların bir arada kullanıldığını göstermek için Şekil 4.7’de gösterilen kodlar kurgulanmıştır. Bu kodlar kullanıcılar tarafından özelleştirilerek kendi web projelerinde kullanılmak üzere geliştirilebilir.

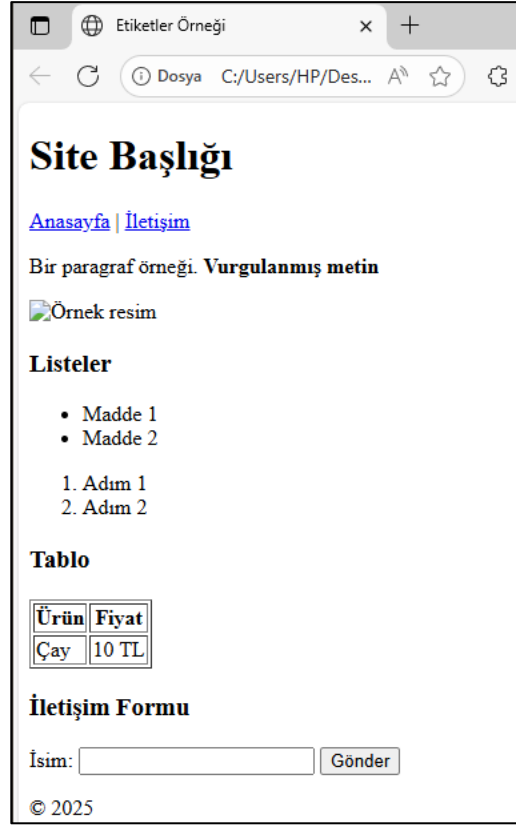


Şekil 4.7. Temel HTML Kodlarının Kullanıldığı Örnek Bir Kodlama

Şekil 4.7’deki kodlar *Visual Studio Code* editöründe çalıştırıldığında veya bir not defterine yapıştırılıp uzantısı *\*.html* olarak kaydedilip çalıştırıldığında Şekil 4.8’deki gibi bir sayfa tasarımıyla karşılaşılacaktır.



Web projelerinde Visual Studio Code editörü yaygın olarak kullanılmaktadır



Şekil 4.8. Temel HTML Kodlarının Sonuçları

## Semantik ve Erişilebilirlik

Semantik HTML, etiketlerin yapı ve anlamına göre doğru şekilde kullanılmasını sağlayarak hem geliştiricilere hem de arama motorlarına sayfa içeriğini daha anlaşılır kılar. Erişilebilirlik ise, web sitelerinin engelli kullanıcılar dâhil herkes tarafından rahatça kullanılabilmesini hedefler. Doğru semantik ve erişilebilir etiketler kullanmak hem SEO açısından avantaj sağlar hem de tüm kullanıcılar için eşit bir deneyim sunar. HTML’de semantik ve erişilebilirlik açısından önemli olan bazı etiketler örneklerle birlikte aşağıdaki gibi sunulmaktadır:



Web projenizin optimizasyonunda semantik etiketler büyük önem taşır.

### <header> Etiketi

Sayfanın veya bir bölümün başlık alanını belirtmek için kullanılır. Genellikle logo, site başlığı veya ana navigasyonu içinde bulundurur. Semantik olduğu için hem geliştiricilerin sayfayı anlamasını hem de ekran okuyucuların bölümleri tanımasını kolaylaştırır. Örnek kod kullanımı Şekil 4.9’daki gibi sunulmaktadır.

```
<header>
  <h1>Web Siteme Hoşgeldiniz</h1>
  <nav>
    <a href="#hakkinda">Hakkında</a>
    <a href="#iletisim">İletişim</a>
  </nav>
</header>
```

Şekil 4.9. <header> Etiketi Kullanımı

## <nav> Etiketi

Sayfa içindeki ana bağlantı menülerini tanımlamak için kullanılır. Semantik bir etikettir ve ekran okuyucular “bu bir menüdür” diyerek kullanıcıya yardımcı olur. Birden fazla <nav> etiketi farklı menüler için kullanılabilir. Şekil 4.10’da <nav> etiketinin kullanımı görülebilir:

```
<nav>
  <ul>
    <li><a href="#anasayfa">Anasayfa</a></li>
    <li><a href="#urunler">Ürünler</a></li>
    <li><a href="#blog">Blog</a></li>
  </ul>
</nav>
```

Şekil 4.10. <nav> Etiketi Kullanımı

## <section> Etiketi

Belirgin, anlamlı bir bölümü veya konuyu tanımlamak için kullanılmaktadır. Birden fazla tematik parçayı birbirinden ayırmak için kullanılır ve her bölümde başlık bulunması önerilir. Şekil 4.11’de <section> etiketinin kullanımı görülebilir:

```
<section>
  <h2>Hizmetlerimiz</h2>
  <p>Web tasarımı, yazılım geliştirme ve danışmanlık hizmetleri sunuyoruz.</p>
</section>
```

Şekil 4.11. <section> Etiketi Kullanımı

## <article> Etiketi

Bağımsız, taşınabilir içerik parçalarını tanımlar (ör. blog gönderisi, haber makalesi). Kendi başına anlamlıdır ve diğer sitelerde de bağımsız olarak kullanılabilir. Ekran okuyucular için içerik bloklarının kolayca ayrışmasını sağlar. Şekil 4.12’de <article> etiketinin kullanımı görülebilir:

```
<article>
  <h2>Duyuru</h2>
  <p>Yeni bir etkinlik düzenliyoruz. Detaylar için sitemizi ziyaret edin!</p>
</article>
```

Şekil 4.12. <article> Etiketi Kullanımı

## <aside> Etiketi

Ana içeriğin dışında tamamlayıcı veya yan içerikler için kullanılır. Reklam, yan menü, ilgili bağlantılar gibi içerikleri kapsar. Şekil 4.13’te <aside> etiketinin kullanımı görülebilir:



<aside> etiketi sayesinde web projenize yan alanlar oluşturabilirsiniz.

```
<aside>
  <h3>İpuçları</h3>
  <ul>
    <li>HTML'de etiketleri doğru kullanın.</li>
    <li>Kodunuzu düzenli yazın.</li>
  </ul>
</aside>
```

Şekil 4.13. <aside> Etiketi Kullanımı

## <footer> Etiketi

Sayfa ya da bir bölümün alt bilgisini tanımlar. Genellikle iletişim, telif hakkı veya ek bağlantılar içermektedir. Semantik olması sayesinde ekran okuyucular tarafından kolayca tanımlanır. Şekil 4.14'te <aside> etiketinin kullanımı görülebilir:

```
<footer>
  <p>&copy; 2025 Tüm hakları saklıdır. Bize ulaşın: info@example.com</p>
</footer>
```

Şekil 4.14. <footer> Etiketi Kullanımı

## <form> Etiketi

Form elemanlarını tanımlayan ve kullanıcıya hangi alanı dolduracağını gösteren etikettir. for özniteliği ile <input>un id'si eşleştirilerek ekran okuyucularda etkileşim artar. Kullanıcılar ve erişilebilirlik araçları için formun anlaşılmasını kolaylaştırır. Şekil 4.15'te form etiketinin kullanımı görülebilir:

```
<form>
  <label for="email">E-posta Adresiniz:</label>
  <input type="email" id="email" name="email">
</form>
```

Şekil 4.15. <form> etiketinin kullanımı

Tüm bu etiketler ve öznitilikler, hem okunabilir, düzenli kodlar yazılmasını hem de web sitesinin farklı kullanıcılar tarafından erişilebilir olmasını sağlar. Semantik kodlar ve diğer bütün kodlar Visual Studio Code veya başka bir editör yardımıyla test edilebilir.



for özniteliği ile <input>un id'si eşleştirilir.



Örnek

- Şekil 4.16'de yer alan form elemanlarını ve temel HTML komutlarına ait kodları kullanarak Şekil 4.17'deki forma benzer bir iletişim formu tasarlayabilirsiniz. Şekil 4.17'de formun görüntüsü ek olarak kullanılacak CSS veya JavaScript kodlarına göre biçimsel farklılıklar gösterebilir. Ayrıca kullanılan tarayıcıya göre de bazı değişiklikler gözlenebilir.

```

<!DOCTYPE html>
<html lang="tr">
<head>
  <meta charset="UTF-8">
  <title>Basit İletişim Formu</title>
</head>
<body>
  <h2>İletişim Formu</h2>
  <form>
    <label for="name">Adınız:</label><br>
    <input type="text" id="name" name="name" required><br><br>

    <label for="email">E-Posta:</label><br>
    <input type="email" id="email" name="email" required><br><br>

    <label for="message">Mesajınız:</label><br>
    <textarea id="message" name="message" rows="5" required></textarea><br><br>

    <button type="submit">Gönder</button>
  </form>
</body>
</html>

```

Şekil 4.16. Temel Bir İletişim Formu Örneği

Şekil 4.17. Temel Bir İletişim Formu Örneği

## <audio>, <video>, <embed> ve <source> Etiketleri

**<audio>** etiketi ses oynatmak için kullanılır ve *src* ya da içinde yer alan **<source>** etiketleriyle dosya belirtilir. Bu etikete ait *controls*, *autoplay*, *loop* ve *muted* gibi öznitelikleri vardır. **<video>** etiketi ise video içeriği göstermek için kullanılır. Bu etiket *poster*, *controls*, *width/height* gibi öznitelikler ile birlikte çalışır ve yine farklı formatlar için **<source>** etiketleri içerebilir.

**<source>** etiketi **<audio>** veya **<video>** içinde tarayıcıya sunulan birden fazla formatlı medya kaynağını belirtir ve tarayıcı uygun olan ilk kaynağı seçer. **<embed>** ise PDF, üçüncü taraf oynatıcılar veya gömülü medya gibi harici içerikleri sayfaya



<audio>, <video>, <embed> ve <source> gibi medya etiketlerini kullanarak web projenizi zenginleştirebilirsiniz.

yerleştirmek için kullanılan genel amaçlı bir etikettir. Bu etiketler birlikte kullanıldığında çoklu format uyumluluğu, altyazı/altyazı desteği için **<track>** ve **erişilebilirlik** için uygun etiketleme önem taşımaktadır.

Şekil 4.18’de **Visual Studio Code** editöründe bu elamanların kullanımına ilişkin örnek bir uygulama görülmektedir. Kodlara ait kaynaklar örnek olarak sunulmuştur. Başka bir ifadeyle **<source>** etiketine ait referans kaynakları güncellenerek kaynak adreslerini değiştirebilirler.

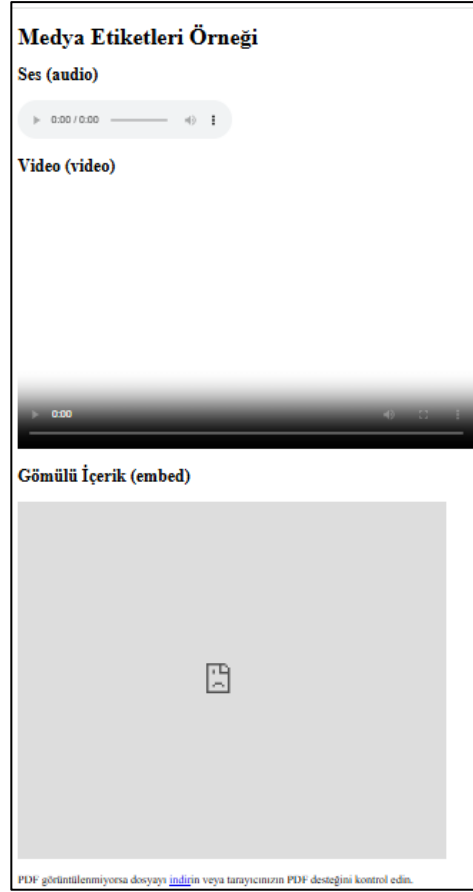
```

web.html > html > body
1  <!DOCTYPE html>
2  <html lang="tr">
3  <head>
4    <meta charset="utf-8" />
5    <title>Medya Etiketleri Örneği</title>
6  </head>
7  <body>
8    <h1>Medya Etiketleri Örneği</h1>
9
10   <!-- Audio örneği: birden fazla source ve erişilebilirlik için aria-label -->
11   <section>
12     <h2>Ses (audio)</h2>
13     <audio controls aria-label="Örnek ses çalar">
14       <source src="audio/example.mp3" type="audio/mpeg">
15       <source src="audio/example.ogg" type="audio/ogg">
16       Tarayıcınız audio etiketini desteklemiyor. <a href="audio/example.mp3">Buradan indir</a>.
17     </audio>
18   </section>
19
20   <!-- Video örneği: poster, multiple sources, altyazı (track) -->
21   <section>
22     <h2>Video (video)</h2>
23     <video controls width="640" height="360" poster="video/poster.jpg" aria-label="Örnek video oynatıcı">
24       <source src="video/example.webm" type="video/webm">
25       <source src="video/example.mp4" type="video/mp4">
26       <!-- Türkçe altyazı örneği -->
27       <track kind="subtitles" srclang="tr" label="Türkçe" src="video/subtitles-tr.vtt" default>
28       Tarayıcınız video etiketini desteklemiyor. <a href="video/example.mp4">Buradan indir</a>.
29     </video>
30   </section>
31
32   <!-- Embed örneği: PDF veya harici içerik gömme -->
33   <section>
34     <h2>Gömülü İçerik (embed)</h2>
35     <!-- PDF gömme örneği; type belirtmek faydalıdır -->
36     <embed src="docs/example.pdf" type="application/pdf" width="600" height="500">
37     <p>PDF görüntülenmiyorsa dosyayı <a href="docs/example.pdf">indir</a>ın veya tarayıcınızın PDF desteğini kontrol edin.</p>
38   </section>
39 </body>
40 </html>

```

Şekil 4.18. <audio>, <video>, <embed> ve <source> Etiketlerinin Kullanımı

Bu kodlar çalıştırıldığında Şekil 4.19’daki gibi bir önizleme ile karşılaşılacaktır. Elde edilen bu bileşenlere ait öznitelikler (genişlik, yükseklik değerleri, dosya türü vs.) değiştirilerek kullanıcıların isteğine göre özelleştirilebilir.



Şekil 4.19. Kodlara Ait Önizleme Sonuçları



Tutarlılık, farklı cihazlarda aynı deneyimi sunma ilkesidir.



## Bireysel Etkinlik

- Kâğıt ve kalem kullanarak kendinize ait bir web sayfası tasarlayınız.
- Tasarladığınız web sayfasının iskeletini HTML kodları kullanarak ön-izlemesini yapınız.
- <header>, <nav>, <main>, <footer> gibi semantik HTML etiketlerini bu aşamada kullanabilirsiniz.
  - Web sayfanıza Liste, tablo, form ve görsel eklemeler (ör: <ul>, <table>, <form>, <img>) yapınız.
  - Web sayfanıza bağlantı etiketlerini kullanarak (<a href="...">), iç içe liste yapma ve sayfa içi bağlantılar oluşturunuz.
  - Basit CSS kodlarını HTML içinde inline veya <style> etiketiyle ekleyerek sayfa tasarımınızı özelleştiriniz.
- Hazırladığınız web projesine ilişkin kaynak kodlarınızı bir editörde (Visual Studio Code, NotePad++ gibi) test ediniz.

## SONUÇ VE DEĞERLENDİRME

Web tasarımı, günümüz dijital dünyasında kullanıcıların ihtiyaçlarına uygun, erişilebilir ve estetik açıdan tatmin edici platformlar oluşturma'nın temelini oluşturmaktadır. Bu bağlamda, HTML, CSS ve JavaScript gibi temel teknolojilerin bilinmesi ve etkin kullanımı, başarılı bir web sitesi geliştirmek için vazgeçilmezdir. Ayrıca, semantik HTML etiklerinin kullanımı, sitenin erişilebilirliğini ve arama motorlarındaki görünürlüğünü artırırken, kullanıcı deneyimini de iyileştirir. Modern tasarım ilkeleri ve araçlarıyla uyumlu, mobil uyumlu ve hızlı yüklenebilir web sayfaları tasarlamak, rekabet ortamında öne çıkmak için kritik öneme sahiptir. Bu süreçte, tasarım ve geliştirme aşamalarında kullanılan araçların ve teknolojilerin sürekli güncellenmesi ve yeniliklere uyum sağlanması, sürdürülebilir ve etkili web projeleri ortaya koymayı sağlar.

Güncel web tasarım trendleri ve teknolojileri göz önüne alındığında, erişilebilirlik ve kullanıcı odaklı tasarımın önemi her geçen gün artmaktadır. Yapay zekâ destekli araçlar ve otomasyon çözümleri, tasarım sürecini hızlandırırken, daha yenilikçi ve kullanıcı dostu arayüzlerin geliştirilmesine imkân tanımaktadır. Ayrıca, responsive tasarım ilkeleri sayesinde farklı cihazlarda tutarlı ve fonksiyonel kullanıcı deneyimleri sunmak, modern web geliştirme standartlarının temelini oluşturmaktadır. Bu nedenle, web tasarımı alanında uzmanlaşmak ve güncel teknolojileri takip etmek hem tasarım kalitesini artırmak hem de kullanıcı memnuniyetini sağlamak adına büyük önem taşımaktadır. Sonuç olarak, web tasarımı, teknik bilgi ve estetik anlayışın birleştiği disiplinlerarası bir alan olup, sürekli gelişen teknolojilere uyum sağlama yeteneği, başarının anahtarıdır.



## Özet

- Web geliştirme, günümüzde dijital ortamların temelini oluşturan ve sürekli yenilenen önemli bir alandır. Bu alanda iki ana bölüm bulunur: Front-end ve Back-end. Front-end, kullanıcıların doğrudan etkileşimde bulunduğu ve görsel olarak deneyimlediği bölümdür. Bu bölümde kullanılan temel teknolojiler HTML, CSS ve JavaScript'tir. HTML (HyperText Markup Language), web sayfalarının yapı taşlarını oluşturur ve içeriklerin düzenli, anlamlı ve erişilebilir olmasını sağlar. HTML5 ile birlikte gelen yeni semantik etiketler, sayfanın yapısını daha anlamlı hale getirir ve erişilebilirliği artırır. Örneğin, <header>, <nav>, <section>, <article>, <aside>, <footer> gibi etiketler, içeriklerin anlamını belirginleştirir ve arama motorları ile erişim araçlarının sayfayı daha iyi anlamasına olanak tanır.
- HTML dokümanları oluştururken, sayfanın temel yapısını belirlemek için <!DOCTYPE html> ifadesi kullanılır. Bu ifade, tarayıcıya sayfanın HTML5 standardında olduğunu belirtir. Ayrıca, sayfanın dilini belirlemek için <html lang="tr"> gibi etiketler tercih edilir. Bu, özellikle Türkçe içeriklerde dil ayarını yapar ve erişilebilirlik açısından önemlidir. Karakter kodlaması ise <meta charset="utf-8"> etiketi ile ayarlanır; bu, özellikle Türkçe gibi karakterlerin doğru görüntülenmesini sağlar ve karakter sorunlarının önüne geçer. Sayfanın stil ve tasarımını şekillendirmek için CSS kullanılır. CSS dosyası, <link rel="stylesheet" href="stil.css"> etiketiyle HTML sayfasına bağlanır. Bu sayede, sayfanın görünüm ve düzeni kontrol edilir. JavaScript ise sayfaya dinamiklik ve etkileşim kazandırmak için kullanılır ve <script src="script.js"></script> etiketiyle çağrılır. HTML5 ile birlikte, <script> etiketinde type özelliğine gerek kalmamıştır, bu da kodların daha sade olmasını sağlar.
- Web tasarım ve geliştirme sürecinde, çeşitli araçlar büyük kolaylık sağlar. Adobe Photoshop, görsel içeriklerin hazırlanmasında ve düzenlenmesinde kullanılırken, Figma özellikle tasarım aşamasında ekipler arası iş birliği ve prototip oluşturma için tercih edilir. Kod yazarken ise Visual Studio Code gibi gelişmiş kod editörleri, kodlama sürecini hızlandırır ve hata oranını azaltır. Ayrıca, bu araçlar ile kodlama sırasında otomatik tamamlama ve hata kontrolü gibi özellikler kullanılır.
- Medya içeriklerini web sayfalarına eklemek de oldukça yaygındır. Video ve ses dosyalarını sayfaya entegre etmek için <video>, <audio>, <embed> ve <source> gibi etiketler kullanılır. Örneğin, <video> etiketi ile videolar gömülebilir ve kontrol edilmesi sağlanabilir. Ses eklemek için ise <audio> etiketi tercih edilir. Kaynak dosyalarını belirtmek için <source src="ses.mp3"> gibi yapılar kullanılır. Ayrıca, dışarıdan içerik veya medya eklerken <iframe> etiketi de sıkça tercih edilir, özellikle başka web sayfalarını gömme işlemlerinde kullanılır. Bu, özellikle harita, video veya diğer içeriklerin gömülmesi için oldukça kullanışlıdır.
- Etkileşimli web sayfaları tasarlamak için form elemanları ve JavaScript kullanılır. Bu sayede kullanıcıların giriş yapması, seçim yapması veya veri göndermesi gibi işlemler gerçekleştirilebilir. Formlar aracılığıyla alınan veriler, sunucuya iletilerek dinamik içerik ve işlemler yapılabilir. Bu, özellikle kullanıcı kaydı, giriş, geri bildirim veya alışveriş gibi işlemler için önemlidir.
- Sonuç olarak, modern web geliştirme sürecinde temel olarak HTML, CSS ve JavaScript kullanılır. HTML sayfanın iskeletini oluştururken, CSS görsel ve tasarım özelliklerini belirler. JavaScript ise sayfaya hareket ve etkileşim kazandırır. Bu temel teknolojilere ek olarak, çeşitli araçlar ve standartlar, web sayfalarının hem görsel açıdan çekici hem de fonksiyonel ve erişilebilir olmasını sağlar. Günümüzde, bu temel bilgilerle birlikte yeni teknolojiler ve standartlar sürekli gelişmekte ve güncellenmektedir. Bu nedenle, web geliştirme alanında çalışanların, yenilikleri takip etmeleri ve kendilerini sürekli geliştirmeleri büyük önem taşır.

## DEĞERLENDİRME SORULARI

1. HTML belgesinin en üst satırında hangi ibare bulunmalıdır?
  - a) <html>
  - b) <!DOCTYPE html>
  - c) <head>
  - d) <body>
  - e) <meta>
2. Aşağıdakilerden hangisi HTML etiketleri arasında yer almaz?
  - a) <header>
  - b) <footer>
  - c) <style>
  - d) <article>
  - e) <content>
3. Bir sayfanın başlık bilgisi hangi etiket içinde bulunur?
  - a) <body>
  - b) <footer>
  - c) <h1>
  - d) <meta>
  - e) <title>
4. Aşağıdaki etiketlerden hangisi bir görsel eklemek için kullanılır?
  - a) <video>
  - b) <audio>
  - c) <img>
  - d) <link>
  - e) <canvas>
5. HTML’de sırasız liste oluşturmak için hangi etiket kullanılır?
  - a) <ol>
  - b) <ul>
  - c) <li>
  - d) <table>
  - e) <div>
6. HTML’de başlık seviyelerini belirtmek için hangi etiketler kullanılır?
  - a) <h1> - <h3>
  - b) <h1> - <h6>
  - c) <heading>
  - d) <header> - <footer>
  - e) <title> - <header>

7. Bir form elemanını tanımlamak için hangi etiket kullanılır?
  - a) <input>
  - b) <button>
  - c) <form>
  - d) <textarea>
  - e) <html>
  
8. Aşağıdakilerden hangisi kullanıcıdan veri toplamak için kullanılan bir HTML etiketidir?
  - a) <img>
  - b) <form>
  - c) <div>
  - d) <span>
  - e) <figure>
  
9. Semantik HTML nedir?
  - a) HTML'in hızlı yazılması.
  - b) Yalnızca görsel öğeleri içermesi.
  - c) Tasarımın estetik açıdan güzel olması.
  - d) Etiketlerin yapı ve anlamına göre doğru kullanılması.
  - e) Yalnızca metin içermesi.
  
10. Responsive (duyarlı) tasarımın amacı nedir?
  - a) Sadece masaüstü kullanıcıları için tasarım yapmak.
  - b) Farklı cihazlarda uyumlu görünüm sağlamak.
  - c) Sadece mobil cihazlar için tasarım yapmak.
  - d) Hızlı yüklenen sayfalar oluşturmak.
  - e) Görsel öğeleri kaldırmak.

**Cevap Anahtarı**

1.b, 2.e, 3.e, 4.c, 5.b, 6.b, 7.e, 8.b, 9.d, 10.b

**YARARLANILAN KAYNAKLAR**

- [1] McFedries, P. (2024). Web Design Playground. Simon and Schuster.
- [2] Safudin, M., & Setiaji, S. (2025). Design and Development of a Web-Based Library Application for Sawah Subdistrict, South Tangerang. Jurnal Multidisiplin Sahombu, 5(5), 1550-1564.
- [3] Palmer, Z. B., & Oswal, S. K. (2025). Constructing websites with generative AI tools: The accessibility of their workflows and products for users with disabilities. Journal of Business and Technical Communication, 39(1), 93-114.
- [4] Dean, J. (2025). Web Programming with HTML, CSS, and JavaScript 2E Print on Demand. Jones & Bartlett Learning.